

```

//+-----
//|                                     RectangleZoneAnalyzer_04.mq4
//|
//|  Блок 04: отбор и ранжирование статистических паттернов.
//|
//|  Вход : age_statistics.csv
//|         pattern_statistics.csv
//|
//|  Выход: pattern_ranking.csv
//|         pattern_rejected.csv
//|
//|  Логика отбора:
//|  1. Минимальная выборка и число определившихся исходов.
//|  2. Нижние границы Уилсона для локального и структурного
//|     разворота.
//|  3. Улучшение относительно более простого родительского
//|     паттерна.
//|  4. Удаление точных дублей.
//|  5. Рейтинг по надёжности, преимуществу, выборке и MFE/MAE.
//+-----
#property copyright "Copyright 2026"
#property version  "1.00"
#property strict
#property script_show_inputs

//--- Входные и выходные файлы находятся в MQL4\Files
input string InputAgeStatisticsFileName =
"age_statistics.csv";
input string InputPatternStatisticsFileName =
"pattern_statistics.csv";
input string OutputRankingFileName =
"pattern_ranking.csv";

```

```

input string OutputRejectedFileName          =
"pattern_rejected.csv";

//--- Обычно файлы относятся к одному графику.
input bool   FilterCurrentChartOnly          = true;

//--- По умолчанию оставляем торговый возраст и отдельные BULL/BEAR
//--- Это устраняет дубли CALENDAR_TIME/TRADING_TIME и ALL/BULL/BEAR
input bool   UseTradingTimeOnly               = true;
input bool   UseZoneSpecificOnly              = true;

//--- Минимальная статистическая устойчивость
input int    MinSamples                      = 100;
input int    StrongSamples                   = 300;
input int    MinLocalDecisiveSamples         = 80;
input int    MinStructuralDecisiveSamples    = 80;

//--- Минимальные нижние 95%-границы Уилсона
//--- Паттерн проходит, если выполнен хотя бы один из двух порогов.
input double MinLocalWilson                  = 25.0;
input double MinStructuralWilson             = 50.0;

//--- Минимальное улучшение относительно родителя, процентных пунктов
//--- Паттерн проходит, если локальное ИЛИ структурное улучшение
//--- не меньше этого значения.
input double MinImprovementPctPoints        = 2.0;

//--- Необязательный фильтр MFE/MAE. 0 отключает.
input double MinMfeMaeRatio                  = 0.0;

//--- Веса итогового рейтинга. Нормализуются автоматически.
input double WeightLocalWilson               = 0.35;
input double WeightStructuralWilson          = 0.40;
input double WeightMfeMae                    = 0.10;
input double WeightImprovement               = 0.15;

//--- Максимальное число строк рейтинга. 0 = сохранить все.
input int    MaxRankingRows                   = 0;

//--- Печать прогресса; 0 отключает сообщения.
input int    ProgressEveryRows                = 1000;

#define STAT_CSV_COLUMNS 77

//+-----

```

```
//| Одна строка входной статистики
```

```
//+-----
```

```
struct PatternRow
{
    string patternType;
    string ageBasis;
    string symbol;
    string timeframe;
    string zoneType;
    string ageBucket;
    string touchBucket;
    string depthBucket;
    string approachBucket;
    string heightBucket;
    string sessionBucket;

    int     samples;
    int     localDecisive;
    int     structuralDecisive;

    double  localReversalPct;
    double  localWilson;
    double  structuralOppositePct;
    double  structuralWilson;

    double  avgMFE_CloseATR;
    double  avgMAE_CloseATR;
    double  avgMfeMaeRatio;
    double  avgBarsPrimaryReversal;
    double  avgBarsLocalBreakout;
    double  avgBarsStructuralReversal;
    double  avgBarsStructuralBreakout;

    bool     isAgeBaseline;
};
```

```
//+-----
```

```
//| Строка результата отбора
```

```
//+-----
```

```
struct RankedPattern
{
    int     sourceIndex;
    int     parentIndex;

    bool     accepted;
    string   rejectReason;
    string   selectionReason;
};
```

```

    string reliabilityTier;

    double parentLocalWilson;
    double parentStructuralWilson;
    int     parentSamples;

    double localImprovementPP;
    double structuralImprovementPP;

    double sampleFactor;
    double payoffScore;
    double improvementScore;
    double finalScore;

    int     rank;
};

PatternRow    g_rows[];
RankedPattern g_results[];

//--- Индексы нужных колонок
int g_idxPatternType      = -1;
int g_idxAgeBasis         = -1;
int g_idxScopeSymbol     = -1;
int g_idxScopeTimeframe  = -1;
int g_idxZoneType         = -1;
int g_idxAgeBucket        = -1;
int g_idxTouchBucket      = -1;
int g_idxDepthBucket      = -1;
int g_idxApproachBucket   = -1;
int g_idxZoneHeightBucket = -1;
int g_idxSessionBucket    = -1;
int g_idxSamples          = -1;
int g_idxLocalDecisiveSamples = -1;
int g_idxLocalReversalPctDecisive = -1;
int g_idxLocalReversalWilsonLower95 = -1;
int g_idxStructuralDecisiveSamples = -1;
int g_idxStructuralOppositePctDecisive = -1;
int g_idxStructuralOppositeWilson95 = -1;
int g_idxAvgMFE_CloseATR  = -1;
int g_idxAvgMAE_CloseATR  = -1;
int g_idxAvgMFE_MAE_Ratio = -1;
int g_idxAvgBarsPrimaryReversal = -1;
int g_idxAvgBarsLocalBreakout = -1;
int g_idxAvgBarsStructuralReversal = -1;
int g_idxAvgBarsStructuralBreakout = -1;

int g_rowsReadAge      = 0;
int g_rowsReadPattern  = 0;
int g_acceptedCount    = 0;
int g_rejectedCount    = 0;

```

```
//+-----  
//| Имя текущего таймфрейма  
//+-----  
string TimeframeToString(int timeframe)  
{  
    switch(timeframe)  
    {  
        case PERIOD_M1: return("M1");  
        case PERIOD_M5: return("M5");  
        case PERIOD_M15: return("M15");  
        case PERIOD_M30: return("M30");  
        case PERIOD_H1: return("H1");  
        case PERIOD_H4: return("H4");  
        case PERIOD_D1: return("D1");  
        case PERIOD_W1: return("W1");  
        case PERIOD_MN1: return("MN1");  
    }  
  
    return(IntegerToString(timeframe));  
}  
  
//+-----  
//| Ограничение значения  
//+-----  
double ClampDouble(double value, double minimum, double maximum)  
{  
    if(value < minimum)  
        return(minimum);  
  
    if(value > maximum)  
        return(maximum);  
  
    return(value);  
}  
  
//+-----  
//| Безопасное чтение целого  
//+-----  
int ParseInt(string value)  
{  
    if(StringLen(value) == 0)  
        return(0);
```

```

    long parsed = StringToInteger(value);

    if(parsed > 2147483647)
        parsed = 2147483647;

    if(parsed < -2147483647)
        parsed = -2147483647;

    return((int)parsed);
}

//+-----

//| Безопасное чтение double

//+-----
double ParseDouble(string value, double defaultValue)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return(StringToDouble(value));
}

//+-----

//| Поиск колонки

//+-----
int FindColumn(string &headers[], string columnName)
{
    int count = ArraySize(headers);

    for(int i = 0; i < count; i++)
    {
        if(headers[i] == columnName)
            return(i);
    }

    return(-1);
}

//+-----

//| Назначение колонок

//+-----
bool MapColumns(string &headers[])
{

```

```

    g_idxPatternType           = FindColumn(headers,
"PatternType");
    g_idxAgeBasis              = FindColumn(headers,
"AgeBasis");
    g_idxScopeSymbol          = FindColumn(headers,
"ScopeSymbol");
    g_idxScopeTimeframe       = FindColumn(headers,
"ScopeTimeframe");
    g_idxZoneType              = FindColumn(headers,
"ZoneType");
    g_idxAgeBucket             = FindColumn(headers,
"AgeBucket");
    g_idxTouchBucket           = FindColumn(headers,
"TouchBucket");
    g_idxDepthBucket           = FindColumn(headers,
"DepthBucket");
    g_idxApproachBucket        = FindColumn(headers,
"ApproachBucket");
    g_idxZoneHeightBucket      = FindColumn(headers,
"ZoneHeightBucket");
    g_idxSessionBucket         = FindColumn(headers,
"SessionBucket");
    g_idxSamples               = FindColumn(headers,
"Samples");
    g_idxLocalDecisiveSamples  = FindColumn(headers,
"LocalDecisiveSamples");
    g_idxLocalReversalPctDecisive = FindColumn(headers,
"LocalReversalPctDecisive");
    g_idxLocalReversalWilsonLower95 = FindColumn(headers,
"LocalReversalWilsonLower95");
    g_idxStructuralDecisiveSamples = FindColumn(headers,
"StructuralDecisiveSamples");
    g_idxStructuralOppositePctDecisive = FindColumn(headers,
"StructuralOppositePctDecisive");
    g_idxStructuralOppositeWilson95 = FindColumn(headers,
"StructuralOppositeWilsonLower95");
    g_idxAvgMFE_CloseATR       = FindColumn(headers,
"AvgMFE_CloseATR");
    g_idxAvgMAE_CloseATR       = FindColumn(headers,
"AvgMAE_CloseATR");
    g_idxAvgMFE_MAE_Ratio      = FindColumn(headers,
"AvgMFE_MAE_Ratio");
    g_idxAvgBarsPrimaryReversal = FindColumn(headers,
"AvgBarsToPrimaryReversal");
    g_idxAvgBarsLocalBreakout  = FindColumn(headers,
"AvgBarsToLocalBreakout");
    g_idxAvgBarsStructuralReversal = FindColumn(headers,
"AvgBarsToStructuralReversal");
    g_idxAvgBarsStructuralBreakout = FindColumn(headers,
"AvgBarsToStructuralBreakout");

    if(g_idxPatternType < 0 ||

```

```

g_idxAgeBasis < 0 ||
g_idxScopeSymbol < 0 ||
g_idxScopeTimeframe < 0 ||
g_idxZoneType < 0 ||
g_idxAgeBucket < 0 ||
g_idxTouchBucket < 0 ||
g_idxDepthBucket < 0 ||
g_idxApproachBucket < 0 ||
g_idxZoneHeightBucket < 0 ||
g_idxSessionBucket < 0 ||
g_idxSamples < 0 ||
g_idxLocalDecisiveSamples < 0 ||
g_idxLocalReversalPctDecisive < 0 ||
g_idxLocalReversalWilsonLower95 < 0 ||
g_idxStructuralDecisiveSamples < 0 ||
g_idxStructuralOppositePctDecisive < 0 ||
g_idxStructuralOppositeWilson95 < 0 ||
g_idxAvgMFE_CloseATR < 0 ||
g_idxAvgMAE_CloseATR < 0 ||
g_idxAvgMFE_MAE_Ratio < 0 ||
g_idxAvgBarsPrimaryReversal < 0 ||
g_idxAvgBarsLocalBreakout < 0 ||
g_idxAvgBarsStructuralReversal < 0 ||
g_idxAvgBarsStructuralBreakout < 0)
{
    Print(
"Ошибка: во входном файле отсутствуют обязательные колонки.");
    return(false);
}

return(true);
}

//+-----
//| Добавление строки во внутренний массив
//+-----
bool AddRow(string &fields[], bool isAgeBaseline)
{
    int size = ArraySize(g_rows);

    if(ArrayResize(g_rows, size + 1) != size + 1)
        return(false);

    g_rows[size].patternType      = fields[g_idxPatternType];
    g_rows[size].ageBasis         = fields[g_idxAgeBasis];
    g_rows[size].symbol           = fields[g_idxScopeSymbol];
    g_rows[size].timeframe        = fields[g_idxScopeTimeframe];
    g_rows[size].zoneType         = fields[g_idxZoneType];
    g_rows[size].ageBucket        = fields[g_idxAgeBucket];

```

```

g_rows[size].touchBucket      = fields[g_idxTouchBucket];
g_rows[size].depthBucket     = fields[g_idxDepthBucket];
g_rows[size].approachBucket  = fields[g_idxApproachBucket];
g_rows[size].heightBucket    = fields[g_idxZoneHeightBucket];
g_rows[size].sessionBucket   = fields[g_idxSessionBucket];

g_rows[size].samples          = ParseInt(fields[
g_idxSamples]);
g_rows[size].localDecisive    = ParseInt(fields[
g_idxLocalDecisiveSamples]);
g_rows[size].structuralDecisive = ParseInt(fields[
g_idxStructuralDecisiveSamples]);

g_rows[size].localReversalPct =
    ParseDouble(fields[g_idxLocalReversalPctDecisive], -1.0);

g_rows[size].localWilson =
    ParseDouble(fields[g_idxLocalReversalWilsonLower95], -1.0
);

g_rows[size].structuralOppositePct =
    ParseDouble(fields[g_idxStructuralOppositePctDecisive], -
1.0);

g_rows[size].structuralWilson =
    ParseDouble(fields[g_idxStructuralOppositeWilson95], -1.0
);

g_rows[size].avgMFE_CloseATR =
    ParseDouble(fields[g_idxAvgMFE_CloseATR], -1.0);

g_rows[size].avgMAE_CloseATR =
    ParseDouble(fields[g_idxAvgMAE_CloseATR], -1.0);

g_rows[size].avgMfeMaeRatio =
    ParseDouble(fields[g_idxAvgMFE_MAE_Ratio], -1.0);

g_rows[size].avgBarsPrimaryReversal =
    ParseDouble(fields[g_idxAvgBarsPrimaryReversal], -1.0);

g_rows[size].avgBarsLocalBreakout =
    ParseDouble(fields[g_idxAvgBarsLocalBreakout], -1.0);

g_rows[size].avgBarsStructuralReversal =
    ParseDouble(fields[g_idxAvgBarsStructuralReversal], -1.0
);

g_rows[size].avgBarsStructuralBreakout =
    ParseDouble(fields[g_idxAvgBarsStructuralBreakout], -1.0
);

g_rows[size].isAgeBaseline = isAgeBaseline;

```

```

    return(true);
}

//+-----

//| Чтение одного файла статистики

//+-----
bool LoadStatisticsFile(string fileName, bool isAgeBaseline)
{
    ResetLastError();

    int handle = FileOpen(
        fileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка открытия ",
            fileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, STAT_CSV_COLUMNS);

    for(int h = 0; h < STAT_CSV_COLUMNS; h++)
    {
        if(FileIsEnding(handle))
        {
            Print("Ошибка: файл ", fileName,
" содержит неполный заголовок.");
            FileClose(handle);
            return(false);
        }

        headers[h] = FileReadString(handle);
    }

    if(!MapColumns(headers))
    {
        FileClose(handle);
        return(false);
    }
}

```

```

}

string fields[];
ArrayResize(fields, STAT_CSV_COLUMNS);

int rowsRead = 0;

while(!FileIsEnding(handle))
{
    for(int c = 0; c < STAT_CSV_COLUMNS; c++)
    {
        if(FileIsEnding(handle) && c == 0)
        {
            fields[c] = "";
            break;
        }

        fields[c] = FileReadString(handle);
    }

    if(StringLen(fields[0]) == 0)
        break;

    if(!AddRow(fields, isAgeBaseline))
    {
        Print("Ошибка памяти при чтении ", fileName);
        FileClose(handle);
        return(false);
    }

    rowsRead++;

    if(ProgressEveryRows > 0 &&
        rowsRead % ProgressEveryRows == 0)
    {
        Print("Блок 04: ", fileName, ", прочитано строк ",
rowsRead);
    }
}

FileClose(handle);

if(isAgeBaseline)
    g_rowsReadAge = rowsRead;
else
    g_rowsReadPattern = rowsRead;

Print("Прочитано ", fileName, ": ", rowsRead, " строк.");
return(true);
}

```

```
//+-----
//| Сравнение базовых координат паттернов
//+-----
bool SameBaseCoordinates(PatternRow &first, PatternRow &second)
{
    return(
        first.ageBasis == second.ageBasis &&
        first.symbol == second.symbol &&
        first.timeframe == second.timeframe &&
        first.zoneType == second.zoneType &&
        first.ageBucket == second.ageBucket
    );
}
```

```
//+-----
//| Поиск AGE_ONLY
//+-----
int FindAgeBaseline(PatternRow &row)
{
    int count = ArraySize(g_rows);

    for(int i = 0; i < count; i++)
    {
        if(!g_rows[i].isAgeBaseline)
            continue;

        if(g_rows[i].patternType != "AGE_ONLY")
            continue;

        if(SameBaseCoordinates(row, g_rows[i]))
            return(i);
    }

    return(-1);
}
```

```
//+-----
//| Поиск AGE_TOUCH для AGE_TOUCH_DEPTH
//+-----
int FindTouchParent(PatternRow &row)
{
    int count = ArraySize(g_rows);
```

```

for(int i = 0; i < count; i++)
{
    if(g_rows[i].patternType != "AGE_TOUCH")
        continue;

    if(!SameBaseCoordinates(row, g_rows[i]))
        continue;

    if(g_rows[i].touchBucket == row.touchBucket)
        return(i);
}

return(-1);
}

```

```

//+-----
//| Поиск AGE_DEPTH для AGE_TOUCH_DEPTH

```

```

//+-----
int FindDepthParent(PatternRow &row)
{
    int count = ArraySize(g_rows);

    for(int i = 0; i < count; i++)
    {
        if(g_rows[i].patternType != "AGE_DEPTH")
            continue;

        if(!SameBaseCoordinates(row, g_rows[i]))
            continue;

        if(g_rows[i].depthBucket == row.depthBucket)
            return(i);
    }

    return(-1);
}

```

```

//+-----
//| Сила родителя для выбора более строгой базы

```

```

//+-----
double ParentStrength(int rowIndex)
{
    if(rowIndex < 0 || rowIndex >= ArraySize(g_rows))
        return(-1.0);

    double local = MathMax(g_rows[rowIndex].localWilson, 0.0);
}

```

```

    double structural = MathMax(g_rows[rowIndex]
.structuralWilson, 0.0);

    return(0.5 * local + 0.5 * structural);
}

//+-----

//| Выбор родительского паттерна

//+-----
int FindParentIndex(PatternRow &row)
{
    int ageParent = FindAgeBaseline(row);

    if(row.patternType != "AGE_TOUCH_DEPTH")
        return(ageParent);

    int touchParent = FindTouchParent(row);
    int depthParent = FindDepthParent(row);

    int bestParent = ageParent;
    double bestStrength = ParentStrength(ageParent);

    double touchStrength = ParentStrength(touchParent);

    if(touchStrength > bestStrength)
    {
        bestStrength = touchStrength;
        bestParent = touchParent;
    }

    double depthStrength = ParentStrength(depthParent);

    if(depthStrength > bestStrength)
    {
        bestParent = depthParent;
    }

    return(bestParent);
}

//+-----

//| Полный ключ строки

//+-----
string PatternKey(PatternRow &row)
{
    return(

```

```

        row.patternType + "|" +
        row.ageBasis + "|" +
        row.symbol + "|" +
        row.timeframe + "|" +
        row.zoneType + "|" +
        row.ageBucket + "|" +
        row.touchBucket + "|" +
        row.depthBucket + "|" +
        row.approachBucket + "|" +
        row.heightBucket + "|" +
        row.sessionBucket
    );
}

```

```

//+-----
//| Идентификатор паттерна для CSV

```

```

//+-----
string PatternID(PatternRow &row)
{
    return(
        row.patternType + " " +
        row.ageBasis + " " +
        row.symbol + " " +
        row.timeframe + " " +
        row.zoneType + " " +
        row.ageBucket + " " +
        row.touchBucket + " " +
        row.depthBucket + " " +
        row.approachBucket + " " +
        row.heightBucket + " " +
        row.sessionBucket
    );
}

```

```

//+-----
//| Проверка наличия более сильного точного дубля

```

```

//+-----
bool HasBetterExactDuplicate(int sourceIndex)
{
    PatternRow current;
    current = g_rows[sourceIndex];
    string key = PatternKey(current);
    int count = ArraySize(g_rows);

    for(int i = 0; i < count; i++)
    {

```

```

        if(i == sourceIndex)
            continue;

        if(g_rows[i].isAgeBaseline)
            continue;

        if(PatternKey(g_rows[i]) != key)
            continue;

        if(g_rows[i].samples > current.samples)
            return(true);

        if(g_rows[i].samples == current.samples && i <
sourceIndex)
            return(true);
    }

    return(false);
}

//+-----

//| Причина положительного отбора

//+-----
string BuildSelectionReason(double localImprovement, double
structuralImprovement)
{
    bool localEdge = localImprovement >= MinImprovementPctPoints
;
    bool structuralEdge = structuralImprovement >=
MinImprovementPctPoints;

    if(localEdge && structuralEdge)
        return("LOCAL_AND_STRUCTURAL_EDGE");

    if(localEdge)
        return("LOCAL_EDGE");

    if(structuralEdge)
        return("STRUCTURAL_EDGE");

    return("NO_EDGE");
}

//+-----

//| Категория надёжности

//+-----

```

```

string BuildReliabilityTier(PatternRow &row, double
localImprovement, double structuralImprovement)
{
    bool bothEdges =
        localImprovement >= MinImprovementPctPoints &&
        structuralImprovement >= MinImprovementPctPoints;

    if(row.samples >= StrongSamples &&
        row.localDecisive >= StrongSamples &&
        row.structuralDecisive >= StrongSamples &&
        bothEdges)
    {
        return("A_STRONG_BOTH");
    }

    if(row.samples >= StrongSamples)
        return("B_STRONG_SAMPLE");

    return("C_RELIABLE");
}

//+-----

//| Расчёт итогового рейтинга

//+-----
void CalculateScore(
    PatternRow &row,
    double localImprovement,
    double structuralImprovement,
    double &sampleFactor,
    double &payoffScore,
    double &improvementScore,
    double &finalScore
)
{
    double strongDenominator = MathMax((double)StrongSamples,
1.0);
    double sampleRatio = MathMin((double)row.samples /
strongDenominator, 1.0);

    sampleFactor = MathSqrt(MathMax(sampleRatio, 0.0));

    // MFE/MAE 0.5 -> 0 баллов, 2.0 и выше -> 100 баллов.
    if(row.avgMfeMaeRatio < 0.0)
        payoffScore = 0.0;
    else
        payoffScore = ClampDouble((row.avgMfeMaeRatio - 0.5) /
1.5, 0.0, 1.0) * 100.0;

    double positiveImprovement =

```

```

        MathMax(localImprovement, 0.0) +
        MathMax(structuralImprovement, 0.0);

    // Суммарные 20 процентных пунктов и выше дают 100 баллов.
    improvementScore = ClampDouble(positiveImprovement / 20.0,
0.0, 1.0) * 100.0;

    double weightSum =
        WeightLocalWilson +
        WeightStructuralWilson +
        WeightMfeMae +
        WeightImprovement;

    if(weightSum <= 0.0)
        weightSum = 1.0;

    double localScore = ClampDouble(row.localWilson, 0.0, 100.0
);
    double structuralScore = ClampDouble(row.structuralWilson,
0.0, 100.0);

    double rawScore =
        (
            WeightLocalWilson * localScore +
            WeightStructuralWilson * structuralScore +
            WeightMfeMae * payoffScore +
            WeightImprovement * improvementScore
        ) / weightSum;

    // Выборка меньше StrongSamples получает умеренный штраф.
    finalScore = rawScore * (0.75 + 0.25 * sampleFactor);
}

//+-----

//| Добавление результата

//+-----
bool AddResult(int sourceIndex)
{
    int size = ArraySize(g_results);

    if(ArrayResize(g_results, size + 1) != size + 1)
        return(false);

    RankedPattern result;
    result.sourceIndex = sourceIndex;
    result.parentIndex = -1;
    result.accepted = false;
    result.rejectReason = "";
    result.selectionReason = "";

```

```

result.reliabilityTier = "";
result.parentLocalWilson = -1.0;
result.parentStructuralWilson = -1.0;
result.parentSamples = 0;
result.localImprovementPP = 0.0;
result.structuralImprovementPP = 0.0;
result.sampleFactor = 0.0;
result.payoffScore = 0.0;
result.improvementScore = 0.0;
result.finalScore = 0.0;
result.rank = 0;

PatternRow row;
row = g_rows[sourceIndex];

if(row.isAgeBaseline || row.patternType == "AGE_ONLY")
{
    result.rejectReason = "BASELINE_NOT_RANKED";
    g_results[size] = result;
    return(true);
}

if(FilterCurrentChartOnly)
{
    if(row.symbol != Symbol() ||
        row.timeframe != TimeframeToString(Period()))
    {
        result.rejectReason = "OTHER_CHART";
        g_results[size] = result;
        return(true);
    }
}

if(UseTradingTimeOnly && row.ageBasis != "TRADING_TIME")
{
    result.rejectReason = "CALENDAR_TIME_DUPLICATE";
    g_results[size] = result;
    return(true);
}

if(UseZoneSpecificOnly && row.zoneType == "ALL")
{
    result.rejectReason = "ALL_ZONE_TYPE_DUPLICATE";
    g_results[size] = result;
    return(true);
}

if(row.samples < MinSamples)
{
    result.rejectReason = "TOO_FEW_SAMPLES";
    g_results[size] = result;
    return(true);
}

```

```

}

if(row.localDecisive < MinLocalDecisiveSamples)
{
    result.rejectReason = "TOO_FEW_LOCAL_DECISIVE";
    g_results[size] = result;
    return(true);
}

if(row.structuralDecisive < MinStructuralDecisiveSamples)
{
    result.rejectReason = "TOO_FEW_STRUCTURAL_DECISIVE";
    g_results[size] = result;
    return(true);
}

if(row.localWilson < 0.0 || row.structuralWilson < 0.0)
{
    result.rejectReason = "MISSING_WILSON_VALUE";
    g_results[size] = result;
    return(true);
}

if(row.localWilson < MinLocalWilson &&
row.structuralWilson < MinStructuralWilson)
{
    result.rejectReason = "WEAK_WILSON";
    g_results[size] = result;
    return(true);
}

if(MinMfeMaeRatio > 0.0 &&
(row.avgMfeMaeRatio < 0.0 || row.avgMfeMaeRatio <
MinMfeMaeRatio))
{
    result.rejectReason = "LOW_MFE_MAE_RATIO";
    g_results[size] = result;
    return(true);
}

if(HasBetterExactDuplicate(sourceIndex))
{
    result.rejectReason = "EXACT_DUPLICATE";
    g_results[size] = result;
    return(true);
}

int parentIndex = FindParentIndex(row);
result.parentIndex = parentIndex;

if(parentIndex < 0)
{

```

```

        result.rejectReason = "PARENT_NOT_FOUND";
        g_results[size] = result;
        return(true);
    }

    PatternRow parent;
    parent = g_rows[parentIndex];

    if(parent.localWilson < 0.0 || parent.structuralWilson < 0.0
)
    {
        result.rejectReason = "PARENT_WILSON_MISSING";
        g_results[size] = result;
        return(true);
    }

    result.parentLocalWilson = parent.localWilson;
    result.parentStructuralWilson = parent.structuralWilson;
    result.parentSamples = parent.samples;

    result.localImprovementPP =
        (row.localWilson - parent.localWilson);

    result.structuralImprovementPP =
        (row.structuralWilson - parent.structuralWilson);

    if(result.localImprovementPP < MinImprovementPctPoints &&
        result.structuralImprovementPP < MinImprovementPctPoints)
    {
        result.rejectReason = "NO_EDGE_OVER_PARENT";
        g_results[size] = result;
        return(true);
    }

    CalculateScore(
        row,
        result.localImprovementPP,
        result.structuralImprovementPP,
        result.sampleFactor,
        result.payoffScore,
        result.improvementScore,
        result.finalScore
    );

    result.accepted = true;
    result.selectionReason = BuildSelectionReason(
        result.localImprovementPP,
        result.structuralImprovementPP
    );

    result.reliabilityTier = BuildReliabilityTier(
        row,

```

```

        result.localImprovementPP,
        result.structuralImprovementPP
    );

    g_results[size] = result;
    return(true);
}

//+-----
//| Оценка всех паттернов
//+-----
bool EvaluatePatterns()
{
    int count = ArraySize(g_rows);

    for(int i = 0; i < count; i++)
    {
        if(!AddResult(i))
        {
            Print("Ошибка памяти при оценке паттернов.");
            return(false);
        }
    }

    return(true);
}

//+-----
//| Сравнение для сортировки
//+-----
bool ShouldComeBefore(RankedPattern &first, RankedPattern &
second)
{
    if(first.accepted != second.accepted)
        return(first.accepted);

    if(first.accepted)
    {
        if(MathAbs(first.finalScore - second.finalScore) >
0.0000001)
            return(first.finalScore > second.finalScore);

        PatternRow firstRow;
        firstRow = g_rows[first.sourceIndex];
        PatternRow secondRow;
        secondRow = g_rows[second.sourceIndex];
    }
}

```

```

        if(firstRow.samples != secondRow.samples)
            return(firstRow.samples > secondRow.samples);

        if(firstRow.structuralWilson !=
secondRow.structuralWilson)
            return(firstRow.structuralWilson >
secondRow.structuralWilson);

        return(firstRow.localWilson > secondRow.localWilson);
    }

    return(StringCompare(first.rejectReason, second.rejectReason
) < 0);
}

//+-----

//| Сортировка результатов

//+-----
void SortResults()
{
    int count = ArraySize(g_results);

    for(int i = 0; i < count - 1; i++)
    {
        int best = i;

        for(int j = i + 1; j < count; j++)
        {
            if(ShouldComeBefore(g_results[j], g_results[best]))
                best = j;
        }

        if(best != i)
        {
            RankedPattern temp;
            temp = g_results[i];
            g_results[i] = g_results[best];
            g_results[best] = temp;
        }
    }

    int rank = 0;

    for(int k = 0; k < count; k++)
    {
        if(g_results[k].accepted)
        {
            rank++;
            g_results[k].rank = rank;
        }
    }
}

```

```

    }
}

//+-----

//| Защита текстового поля CSV

//+-----
string EscapeCsv(string value)
{
    bool needQuotes =
        StringFind(value, ";") >= 0 ||
        StringFind(value, "\"") >= 0 ||
        StringFind(value, "\r") >= 0 ||
        StringFind(value, "\n") >= 0;

    if(!needQuotes)
        return(value);

    StringReplace(value, "\"", "\\\"");
    return("\"" + value + "\"");
}

//+-----

//| Добавление поля в строку

//+-----
void AppendField(string &line, string value)
{
    if(StringLen(line) > 0)
        line += ";";

    line += EscapeCsv(value);
}

//+-----

//| Double или пустое поле

//+-----
string DoubleOrBlank(double value, int digits)
{
    if(value < 0.0)
        return("");

    return(DoubleToString(value, digits));
}

```

```

//+-----
//| Заголовок выходных файлов
//+-----
void WriteOutputHeader(int handle, bool rejectedFile)
{
    string line = "";

    if(!rejectedFile)
        AppendField(line, "Rank");

    AppendField(line, "Status");
    AppendField(line, "RejectReason");
    AppendField(line, "SelectionReason");
    AppendField(line, "ReliabilityTier");
    AppendField(line, "PatternID");
    AppendField(line, "ParentPatternID");
    AppendField(line, "PatternType");
    AppendField(line, "AgeBasis");
    AppendField(line, "Symbol");
    AppendField(line, "Timeframe");
    AppendField(line, "ZoneType");
    AppendField(line, "AgeBucket");
    AppendField(line, "TouchBucket");
    AppendField(line, "DepthBucket");
    AppendField(line, "ApproachBucket");
    AppendField(line, "ZoneHeightBucket");
    AppendField(line, "SessionBucket");
    AppendField(line, "Samples");
    AppendField(line, "LocalDecisiveSamples");
    AppendField(line, "LocalReversalPctDecisive");
    AppendField(line, "LocalReversalWilsonLower95");
    AppendField(line, "StructuralDecisiveSamples");
    AppendField(line, "StructuralOppositePctDecisive");
    AppendField(line, "StructuralOppositeWilsonLower95");
    AppendField(line, "ParentSamples");
    AppendField(line, "ParentLocalWilsonLower95");
    AppendField(line, "ParentStructuralWilsonLower95");
    AppendField(line, "LocalImprovementPctPoints");
    AppendField(line, "StructuralImprovementPctPoints");
    AppendField(line, "AvgMFE_CloseATR");
    AppendField(line, "AvgMAE_CloseATR");
    AppendField(line, "AvgMFE_MAE_Ratio");
    AppendField(line, "AvgBarsToPrimaryReversal");
    AppendField(line, "AvgBarsToLocalBreakout");
    AppendField(line, "AvgBarsToStructuralReversal");
    AppendField(line, "AvgBarsToStructuralBreakout");
    AppendField(line, "SampleFactor");
    AppendField(line, "PayoffScore");
    AppendField(line, "ImprovementScore");

```

```

AppendField(line, "FinalScore");

FileWriteString(handle, line + "\r\n");
}

//+-----

//| Запись одной строки

//+-----
void WriteOutputRow(int handle, RankedPattern &result, bool
rejectedFile)
{
    PatternRow row;
    row = g_rows[result.sourceIndex];

    string line = "";

    if(!rejectedFile)
        AppendField(line, IntegerToString(result.rank));

    AppendField(line, result.accepted ? "ACCEPTED" : "REJECTED"
);
    AppendField(line, result.rejectReason);
    AppendField(line, result.selectionReason);
    AppendField(line, result.reliabilityTier);
    AppendField(line, PatternID(row));

    string parentID = "";

    if(result.parentIndex >= 0)
        parentID = PatternID(g_rows[result.parentIndex]);

    AppendField(line, parentID);
    AppendField(line, row.patternType);
    AppendField(line, row.ageBasis);
    AppendField(line, row.symbol);
    AppendField(line, row.timeframe);
    AppendField(line, row.zoneType);
    AppendField(line, row.ageBucket);
    AppendField(line, row.touchBucket);
    AppendField(line, row.depthBucket);
    AppendField(line, row.approachBucket);
    AppendField(line, row.heightBucket);
    AppendField(line, row.sessionBucket);
    AppendField(line, IntegerToString(row.samples));
    AppendField(line, IntegerToString(row.localDecisive));
    AppendField(line, DoubleOrBlank(row.localReversalPct, 4));
    AppendField(line, DoubleOrBlank(row.localWilson, 6));
    AppendField(line, IntegerToString(row.structuralDecisive));

```

```

AppendField(line, DoubleOrBlank(row.structuralOppositePct, 4
));
AppendField(line, DoubleOrBlank(row.structuralWilson, 6));
AppendField(line, IntegerToString(result.parentSamples));
AppendField(line, DoubleOrBlank(result.parentLocalWilson, 6
));
AppendField(line, DoubleOrBlank(
result.parentStructuralWilson, 6));
AppendField(line, DoubleToString(result.localImprovementPP,
4));
AppendField(line, DoubleToString(
result.structuralImprovementPP, 4));
AppendField(line, DoubleOrBlank(row.avgMFE_CloseATR, 6));
AppendField(line, DoubleOrBlank(row.avgMAE_CloseATR, 6));
AppendField(line, DoubleOrBlank(row.avgMfeMaeRatio, 6));
AppendField(line, DoubleOrBlank(row.avgBarsPrimaryReversal,
4));
AppendField(line, DoubleOrBlank(row.avgBarsLocalBreakout, 4
));
AppendField(line, DoubleOrBlank(
row.avgBarsStructuralReversal, 4));
AppendField(line, DoubleOrBlank(
row.avgBarsStructuralBreakout, 4));
AppendField(line, DoubleToString(result.sampleFactor, 6));
AppendField(line, DoubleToString(result.payoffScore, 4));
AppendField(line, DoubleToString(result.improvementScore, 4
));
AppendField(line, DoubleToString(result.finalScore, 6));

FileWriteString(handle, line + "\r\n");
}

```

//+-----

//| Сохранение рейтинга

//+-----

```
bool SaveRanking()
```

```

{
    ResetLastError();

    int handle = FileOpen(
        OutputRankingFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка создания ",
            OutputRankingFileName,

```

```

        ". Код ошибки: ",
        GetLastError()
    );

    ResetLastError();
    return(false);
}

WriteOutputHeader(handle, false);

int written = 0;
int count = ArraySize(g_results);

for(int i = 0; i < count; i++)
{
    if(!g_results[i].accepted)
        continue;

    if(MaxRankingRows > 0 && written >= MaxRankingRows)
        break;

    WriteOutputRow(handle, g_results[i], false);
    written++;
}

FileFlush(handle);
FileClose(handle);

g_acceptedCount = written;

string fullPath =
    TerminalInfoString(TERMINAL_DATA_PATH) +
    "\\MQL4\\Files\\" +
    OutputRankingFileName;

Print("Файл рейтинга создан: ", fullPath);
return(true);
}

```

//+-----

//| Сохранение отклонённых строк

//+-----

```
bool SaveRejected()
```

```
{
```

```
    ResetLastError();
```

```
    int handle = FileOpen(
```

```
        OutputRejectedFileName,
```

```
        FILE_WRITE | FILE_TXT | FILE_ANSI
```

```

);

if(handle == INVALID_HANDLE)
{
    Print(
        "Ошибка создания ",
        OutputRejectedFileName,
        ". Код ошибки: ",
        GetLastError()
    );

    ResetLastError();
    return(false);
}

WriteOutputHeader(handle, true);

int written = 0;
int count = ArraySize(g_results);

for(int i = 0; i < count; i++)
{
    if(g_results[i].accepted)
        continue;

    WriteOutputRow(handle, g_results[i], true);
    written++;
}

FileFlush(handle);
FileClose(handle);

g_rejectedCount = written;

string fullPath =
    TerminalInfoString(TERMINAL_DATA_PATH) +
    "\\MQL4\\Files\\" +
    OutputRejectedFileName;

Print("Файл отклонений создан: ", fullPath);
return(true);
}

//+-----

//| Проверка параметров

//+-----
bool ValidateInputs()
{
    if(MinSamples < 1)

```

```

{
    Print("Ошибка: MinSamples должен быть больше 0.");
    return(false);
}

if(StrongSamples < MinSamples)
{
    Print(
"Ошибка: StrongSamples должен быть не меньше MinSamples.");
    return(false);
}

if(MinLocalDecisiveSamples < 1 ||
    MinStructuralDecisiveSamples < 1)
{
    Print(
"Ошибка: минимальные decisive-выборки должны быть больше 0.");
    return(false);
}

if(MinLocalWilson < 0.0 || MinLocalWilson > 100.0 ||
    MinStructuralWilson < 0.0 || MinStructuralWilson > 100.0)
{
    Print(
"Ошибка: пороги Уилсона должны находиться в диапазоне 0..100."
);
    return(false);
}

if(MinImprovementPctPoints < 0.0)
{
    Print(
"Ошибка: MinImprovementPctPoints не может быть отрицательным."
);
    return(false);
}

if(MinMfeMaeRatio < 0.0)
{
    Print(
"Ошибка: MinMfeMaeRatio не может быть отрицательным.");
    return(false);
}

if(WeightLocalWilson < 0.0 ||
    WeightStructuralWilson < 0.0 ||
    WeightMfeMae < 0.0 ||
    WeightImprovement < 0.0)
{
    Print(
"Ошибка: веса рейтинга не могут быть отрицательными.");
    return(false);
}

```

```

    if (WeightLocalWilson +
        WeightStructuralWilson +
        WeightMfeMae +
        WeightImprovement <= 0.0)
    {
        Print(
"Ошибка: сумма весов рейтинга должна быть больше 0.");
        return(false);
    }

    if (MaxRankingRows < 0)
    {
        Print(
"Ошибка: MaxRankingRows не может быть отрицательным.");
        return(false);
    }

    return(true);
}

```

//+-----

//| Итог в журнал

//+-----

```

void PrintSummary()
{
    int acceptedAll = 0;
    int rejectedAll = 0;
    int tierA = 0;
    int tierB = 0;
    int tierC = 0;
    int localEdge = 0;
    int structuralEdge = 0;
    int bothEdge = 0;

    int count = ArraySize(g_results);

    for(int i = 0; i < count; i++)
    {
        if(g_results[i].accepted)
        {
            acceptedAll++;

            if(g_results[i].reliabilityTier == "A_STRONG_BOTH")
                tierA++;
            else if(g_results[i].reliabilityTier ==
"B_STRONG_SAMPLE")
                tierB++;
            else
                tierC++;
        }
    }
}

```

```

        if(g_results[i].selectionReason == "LOCAL_EDGE")
            localEdge++;
        else if(g_results[i].selectionReason ==
"STRUCTURAL_EDGE")
            structuralEdge++;
        else if(g_results[i].selectionReason ==
"LOCAL_AND_STRUCTURAL_EDGE")
            bothEdge++;
    }
    else
    {
        rejectedAll++;
    }
}

Print(
"=====");
);
Print("БЛОК 04 ЗАВЕРШЁН");
Print(
"-----");
);
Print("Строк age_statistics           : ", g_rowsReadAge);
Print("Строк pattern_statistics         : ", g_rowsReadPattern);
Print("Принято паттернов всего           : ", acceptedAll);
Print("Записано в ranking                 : ", g_acceptedCount);
Print("Отклонено                         : ", rejectedAll);
Print(
"-----");
);
Print("A_STRONG_BOTH                     : ", tierA);
Print("B_STRONG_SAMPLE                   : ", tierB);
Print("C_RELIABLE                        : ", tierC);
Print(
"-----");
);
Print("LOCAL_EDGE                       : ", localEdge);
Print("STRUCTURAL_EDGE                   : ", structuralEdge);
Print("LOCAL_AND_STRUCTURAL_EDGE         : ", bothEdge);
Print(
"=====");
);
}

```

```

//+-----

```

```

//| Точка входа

```

```

//+-----

```

```

void OnStart()
{
    if(!ValidateInputs())
        return;

```

```

Print("Блок 04: чтение ", InputAgeStatisticsFileName, "...")
);

if(!LoadStatisticsFile(InputAgeStatisticsFileName, true))
    return;

Print("Блок 04: чтение ", InputPatternStatisticsFileName,
"...");

if(!LoadStatisticsFile(InputPatternStatisticsFileName, false
))
    return;

if(ArraySize(g_rows) <= 0)
{
    Print("Ошибка: не загружено ни одной строки статистики."
);
    return;
}

if(!EvaluatePatterns())
    return;

SortResults();

if(!SaveRanking())
    return;

if(!SaveRejected())
    return;

PrintSummary();
}

//+-----

```